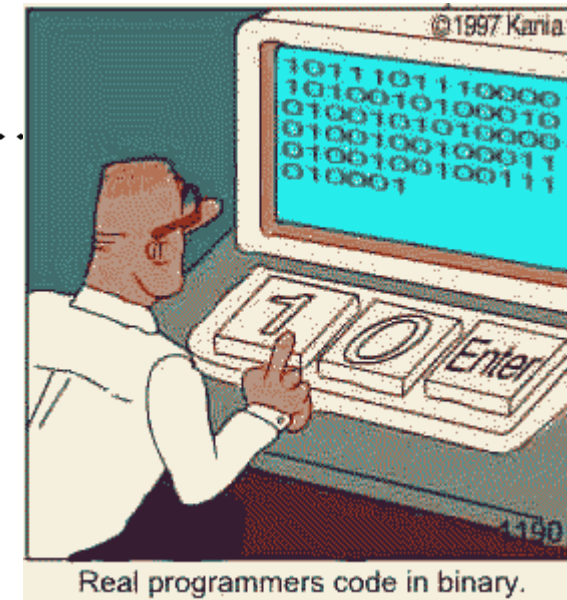


Grundlagen der Programmierung

02 Kleine Einführung



Real programmers code in binary.

- **Wer hat schon Erfahrung im Programmieren?**
- **Wer noch keine?**

- „Den Wecker auf 8:30 stellen.“
- „Die Wiederholung der Simpsons um 3 Uhr nachts automatisch aufnehmen lassen.“
- „Ein Rezept für einen Koch schreiben.“

- **In der Informatik:**
 - „Einer Maschine sagen, was sie selbstständig tun soll.“

- **Die Sprachen der Maschinen sind sehr eingeschränkt (manche Maschinensprachen bestehen aus weniger als 20 Befehlen).**
- **Ändert sich der Prozessor, dann ändert sich auch die Maschinensprache. PCs haben z.B. eine andere Maschinensprache als Macs und ein neuer Pentium hat einen anderen Befehlssatz als ein alter 8086.**
- **=> Maschinen sind ziemlich dumm und man muss wirklich alles explizit sagen.**

- **Schreibe ein Programm zum Braten eines Spiegeleis!**
- **„Ein Ei in einer Pfanne kurz mit etwas Fett braten.“**
- **Jeder menschliche Koch hat damit kein Problem.**
- **Nicht so der Computer...**

So schlau sind Computer nicht...

- Öffne Schrank
- Nimm Pfanne
- Stelle Pfanne auf Herd
- Schalte Herd auf Stufe 3 ein
- Öffne Kühlschrank
- Nimm Eierschachtel
- Öffne Eierschachtel
- Nimm Ei
- Zerschne Ei (ob das wohl gut geht...)
- ...

- **Was soll der Koch tun, wenn keine Pfanne oder keine Eier da sind?**
 - Muss man auch sagen:
 - „Wenn keine Pfanne da, dann Abbruch.“
 - Das kann ganz schön viel Arbeit machen, immer aufzupassen, dass Robustheit und Verlässlichkeit gewährleistet sind.

- **Und wenn wir es vergessen zu sagen, dann steht der Kühlschrank immer noch offen...**

- **Der Computer macht ohne Murren alles, was ihr ihm sagt...**
 - ...und kein bisschen mehr.
- **Sollte also euer Programm nicht laufen, ist es ziemlich sicher eure eigene Schuld.**
- **Man muss sich teilweise eine ganz neue Art des Denkens angewöhnen.**

- **Wie viele Leerzeichen sind im folgenden Satz?**
 - „Dieser Satz enthält 12 Leerzeichen?“
- **Es sind 4.**
- **Aber woher wissen wir das?**
- **Schauen?**
- **Aber wie?**

➤ **Zähler = 0**

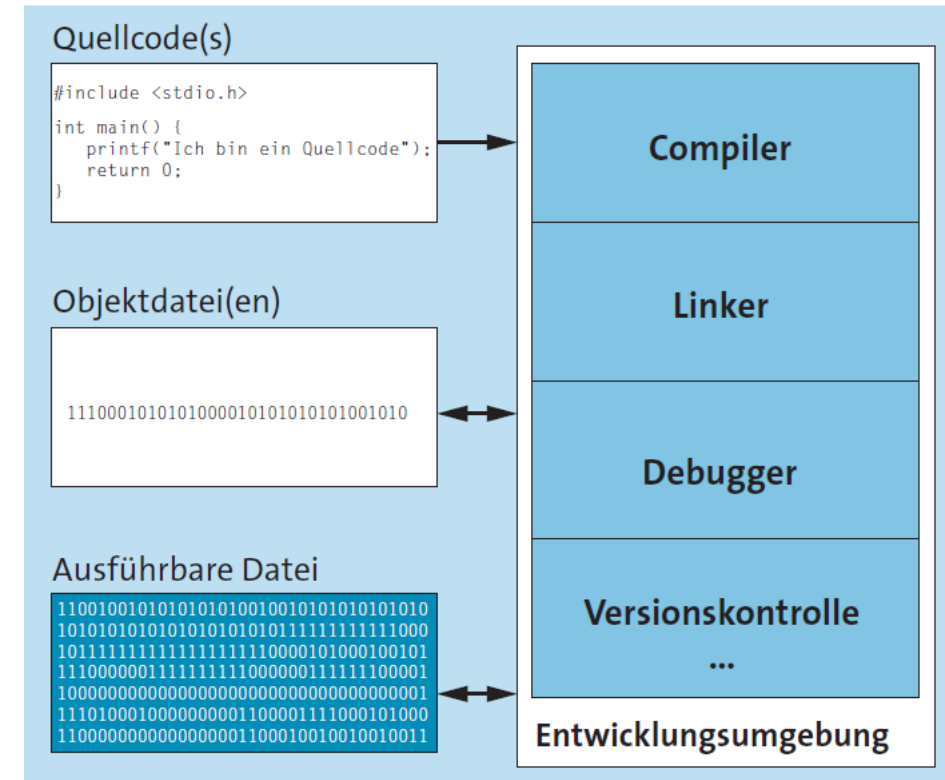
➤ **Nimm jedes Zeichen des Satzes und mache:**

- Falls das Zeichen gleich dem Leerzeichen ist, dann erhöhe Zähler um eins.

- **Was wir gerade gemacht haben, ist einen Algorithmus entwerfen.**
- **Und dann haben wir diesen in Pseudo-Code aufgeschrieben.**
- **Ein **Algorithmus** ist ein Verfahren, wie eine bestimmte Sorte Problem gelöst werden kann (z.B. Leerzeichen zählen).**
 - Beispiele wären: Daten sortieren, Routen planen, Schach spielen...
- **Pseudo-Code ist das umgangssprachliche Ausdrücken eines Algorithmus.**
 - Schon sehr an das Denken der Computer angelehnt, muss aber nicht alle Feinheiten enthalten.

- **Maschinensprache** besteht im Prinzip nur aus einer ganzen Menge Zahlen (Nullen und Einsen).
 - Dies ist für uns Menschen unverständlich und enthält viele Details, die uns nicht interessieren.
 - Deshalb gibt es **Programmiersprachen**, welche näher an unserem Denken liegen, aber vom Computer selbst in Maschinensprache umgewandelt werden können.
 - Im Gegensatz zu Maschinen-Code, welcher in Maschinensprache geschrieben wurde, bezeichnet man Anweisungen an den Computer in einer Programmiersprache als **Quell-Code** (engl. Source) oder **Programm-Code**.
-

FHV



➤ **Quell-Code (5 Programmzeilen, Lines of Code):**

```
1  #include <stdio.h>
2
3  int main(void) {
4      printf("Hello, World!\n");
5      return 0;
6  }
```

- **Beim Start dieses Programms wird "Hello World" ausgegeben und dann das Programm beendet.**

- Diesen Quellcode hat der **Compiler** jetzt in Assembler-Code umgewandelt.
- **Assembler-Code** ist sehr nahe an der Maschinen-sprache, aber (für Eingeweihte) noch lesbar.

Wie viele tausend[!] Zeilen Code sich dahinter verstecken, wollt ihr gar nicht wissen!

```
.file "helloWorld.c"
.version "01.01"
gcc2_compiled.:
.section .rodata
.LC0:
.string "Hello World!\n"
.text
.align 4
.globl main
.type main,@function
main:
    pushl %ebp
    movl %esp,%ebp
    subl $8,%esp
    addl $-12,%esp
    pushl $.LC0
    call printf
    addl $16,%esp
    xorl %eax,%eax
    jmp .L2
    .p2align 4,,7
.L2:
    leave
    ret
.Lfe1:
.size main,.Lfe1-main
.ident "GCC: (GNU) 2.95.4"
```

➤ **Es geht noch 4 Bildschirmseiten so weiter.**

- Nicht unbedingt, wie man sich Programmieren vorstellt.

➤ **"Die Matrix"?**

- Wer will das schon, wenn es auch in 5 lesbaren Zeilen geht...

```
00000000h: 7F 45 4C 46 01 01 01 00 00 00 00 00 00 00 00
00000010h: 02 00 03 00 01 00 00 00 10 83 04 08 34 00 00
00000020h: F8 07 00 00 00 00 00 00 34 00 20 00 06 00 28
00000030h: 1B 00 18 00 06 00 00 00 34 00 00 00 34 80 04
00000040h: 34 80 04 08 C0 00 00 00 C0 00 00 00 05 00 00
00000050h: 04 00 00 00 03 00 00 00 F4 00 00 00 F4 80 04
00000060h: F4 80 04 08 13 00 00 00 13 00 00 00 04 00 00
00000070h: 01 00 00 00 01 00 00 00 00 00 00 00 00 80 04
00000080h: 00 80 04 08 82 04 00 00 82 04 00 00 05 00 00
00000090h: 00 10 00 00 01 00 00 00 84 04 00 00 84 94 04
000000a0h: 84 94 04 08 0C 01 00 00 24 01 00 00 06 00 00
000000b0h: 00 10 00 00 02 00 00 00 98 04 00 00 98 94 04
000000c0h: 98 94 04 08 C8 00 00 00 C8 00 00 00 06 00 00
000000d0h: 04 00 00 00 04 00 00 00 08 01 00 00 08 81 04
000000e0h: 08 81 04 08 20 00 00 00 20 00 00 00 04 00 00
000000f0h: 04 00 00 00 2F 6C 69 62 2F 6C 64 2D 6C 69 6E
00000100h: 78 2E 73 6F 2E 32 00 00 04 00 00 00 10 00 00
00000110h: 01 00 00 00 47 4E 55 00 00 00 00 00 02 00 00
00000120h: 00 00 00 00 1E 00 00 00 03 00 00 00 07 00 00
00000130h: 06 00 00 00 03 00 00 00 05 00 00 00 00 00 00
00000140h: 00 00 00 00 01 00 00 00 02 00 00 00 00 00 00
00000150h: 04 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000160h: 00 00 00 00 00 00 00 00 4B 00 00 00 D0 82 04
00000170h: 27 00 00 00 22 00 00 00 12 00 00 00 E0 82 04
00000180h: 23 00 00 00 22 00 00 00 39 00 00 00 F0 82 04
00000190h: C1 00 00 00 12 00 00 00 0B 00 00 00 00 83 04
000001a0h: 2F 00 00 00 12 00 00 00 2A 00 00 00 70 84 04
000001b0h: 04 00 00 00 11 00 0E 00 61 00 00 00 00 00 00
000001c0h: 00 00 00 00 20 00 00 00 00 6C 69 62 63 2E 73
000001d0h: 2E 36 00 70 72 69 6E 74 66 00 5F 5F 64 65 72
000001e0h: 67 69 73 74 65 72 5F 66 72 61 6D 65 5F 69 6E
000001f0h: 6F 00 5F 49 4F 5F 73 74 64 69 6E 5F 75 73 65
00000200h: 00 5F 5F 6C 69 62 63 5F 73 74 61 72 74 5F 6D
00000210h: 69 6E 00 5F 5F 72 65 67 69 73 74 65 72 5F 66
00000220h: 61 6D 65 5F 69 6E 66 6F 00 5F 5F 67 6D 6F 6E
00000230h: 73 74 61 72 74 5F 5F 00 47 4C 49 42 43 5F 32
00000240h: 30 00 00 00 02 00 02 00 02 00 02 00 01 00 00
00000250h: 01 00 01 00 01 00 00 00 10 00 00 00 00 00 00
```


- **Nachdem wir uns einen Algorithmus überlegt haben, müssen wir eine Programmiersprache wählen.**
- **Hiervon gibt es ziemlich viele (Hunderte).**
- **Einige Beispiele (in alphabetischer Reihenfolge):**
 - Ada, Basic, C, C#, C++, Cobol, Fortran, Eiffel, Erlang, Haskell, Java, Lisp, Malbolge, ML, Ocaml, Occam, Pascal, Perl, PHP, Pike, Prolog, Python, Ruby, Sather, Scheme, Shell, Smalltalk, Tcl, Visual Basic...

- **Jede Programmiersprache hat Stärken und Schwächen und wurde für bestimmte Zwecke entworfen.**
 - "Manche Sprachen haben 20 Worte für Schnee, andere sind Bürokratendeutsch..."
- **Es gilt, eine passende Wahl zu treffen, je nachdem was man tun will.**
 - "Nicht mit der Kettensäge Rasen mähen."
- **Eine Art, Programmiersprachen zu kategorisieren sind Paradigmen.**

- **Imperative Programmiersprachen verwenden Listen von Befehlen, die der Computer Stück für Stück abarbeiten soll. „Tue dies, dann tu das, jetzt jenes.“**
 - Beispiele: C/C++, Java, Pascal, Python, Perl, Ruby...
- **Funktionale Sprachen sind enger an die Mathematik angelehnt und drücken alles durch Funktionen und deren Ergebnisse aus. „Das Ergebnis ist $f(g(x))$.“**
 - Beispiele: Haskell, Lambda-Kalkül, Ocaml, Scheme...

- **Strukturierte Sprachen gruppieren Programmfunktionalität mittels von Prozeduren.**
 - „Mittels dieser Prozedur wird ein Käsekuchen gebacken.“
 - Beispiele: C, Pascal

- **Objekt-Orientierte Sprachen gruppieren mittels eines Objektes (Daten+Prozedur).**
 - Beispiele: C++, Java, Smalltalk

- Bei **Compiler-Sprachen** wird der Quell-Code einmalig in **Maschinensprache** übersetzt. Der Computer kann sie dann **direkt ausführen**.
 - Beispiele: C/C++, Pascal/Delphi

- **Interpretierte Sprachen** werden während des Programmablaufs **von einem Interpreter ausgewertet**. Hierdurch sind sie meist **langsamer**, aber das **Kompilieren fällt komplett weg** und man ist **nicht durch die Maschinesprache in den Fähigkeiten beschränkt**.
 - Beispiele: Java, Basic, Python, Ruby

- **Paradigmen sind Eigenschaften-Klassen von Programmiersprachen.**
- **Sie helfen dabei, die richtige Programmiersprache für eine bestimmte Aufgabe zu finden.**

➤ **Die Sprachen der Fachgebiete:**

- Hardware nahe Programmierung oder Systemprogrammierung => C/C++
- Künstliche Intelligenz => Lisp, Prolog, Scheme
- Computergrafik => C/C++
- Software Engineering, Datenanalyse => Java, C++, Python
- Simulation und Hochleistungsrechnen => Fortran
- Netzwerke und Administration => Shell, Perl

- **Fehler sind bei der Erstellung von Programmen unvermeidlich.**
- **Auch ihr werdet viele Fehler machen.**
- **Wenn man nicht den richtigen Ansatz wählt, kann man mehr Zeit bei der Fehlersuche verbringen als beim Programmieren selbst.**
- **Es gibt viele Arten von Fehlern.**

- **Manche Fehler kann uns der Compiler schon früh berichten:**
- **Syntax-Fehler sind z.B. alle Fehler, in der Satzstellung. Der Computer ist ziemlich gut darin diese zu erkennen.**
- **Beispiel:**
 - „Öffne die kühlshranktür Nimm Eier.“
 - Kühlshranktür ist klein geschrieben
 - Punkt fehlt vor „Nimm“.
 - Achtung: Der Computer wird euch wahrscheinlich eher sowas sagen wie: „Kenne kühlshranktür nicht“ und „Zwei Verben im Satz ohne ‚und‘ gefunden.“

- **Diese werden dem Computer nicht auffallen. Denn er weiß ja nicht, was ihr vorhabt.**
- **Beispiel:**
 - „Falls Eierschachtel nicht leer, dann Abbruch.“
 - Die Syntax stimmt.
 - Aber eigentlich wollten wir nicht abbrechen

➤ **Die besten Werkzeuge, welche die Softwaretechnik bisher gefunden hat, sind:**

- **Entwurf:** Vor dem Programmieren eine solche Struktur finden, dass das Programm einfach wird
- **Durchsichten (Review):** Jemand anderes als der Autor geht den Quell-Code nach bestimmten Kriterien durch.
- **Tests:** Es wird immer wieder automatisch überprüft, ob der Code das macht, was er soll. „Ist ein Spiegelei dabei rausgekommen?“
- Der Computer kümmert sich um die Syntaxfehler.

- **Konzentriert** arbeiten
 - **Genau** arbeiten - Ein einziges falsches Zeichen macht oft den Unterschied zw. kaputt und Erfolg.
 - **Unterbrechungen vermeiden**
 - Eine aktuelle Zeile nicht in der Mitte abbrechen und nachher weitermachen.
 - **Auftretende Fehler verstehen** und nicht durch Versuch und Irrtum "lösen".
 - Wenn man unsicher wird, ruhig einen Schritt zurücktreten und überprüfen, ob man kapiert was da gerade passiert.
 - **Zusammenarbeiten** (Paarprogrammierung)
-

➤ ... jetzt mal live, falls ihr wollt.

- **Für alle die unter euch, die noch nie programmiert haben, oder sich nicht ganz so sicher fühlen:**
 - Schnappt euch Tutorials aus dem Netz.
 - Investiert Zeit! Programmieren ist eine Fähigkeit, die man als Techniker haben muss.
 - -> Lasst euch nicht von den Mathematikern irritieren.
 - Lernt gemeinsam (erspart Frust, Wissen verteilt sich schneller).
 - Glaubt nicht, dass es reicht, Audio-Lernprogramme oder Vorlesungen zu konsumieren, um Programmieren zu lernen.

- **Für alle die unter euch, für die das alles kalter Kaffee war:**
- Helft anderen.
 - Seid nicht überheblich mit euren Fähigkeiten. Ihr musstet es auch mal lernen.
 - Wenn der andere nicht versteht, dann ist es wahrscheinlich euer Problem und nicht deren.
 - Zeigt, was ihr könnt, und macht bei Programmierwettbewerben mit.